Smart Waitlist

From Personas and Requirements to Use Cases and UML

CS3151 Software Engineering | Dr. Passent Elkafrawy | Effat University

Team: Mohammad Altazi, Omar AlMutairi | Deliverables D1 to D7 (+ optional state machine and system sequence diagram)

Scope reminder: Smart Waitlist is a standalone web application. It cannot enroll students and does not integrate with Banner; staff apply accepted offers in Banner by hand. Banner exchange is by CSV files only.

Item	Content
D1	Use case list (MoSCoW), actor cards, traceability matrix
D2	Two fully dressed use cases + brief descriptions of all 18
D3	Use case diagram
D4	Activity diagram (UC-12 Generate Offer List)
D5	Sequence diagram (UC-12 main success scenario)
D6	Domain model
D7	Consistency checklist and peer-review sheet
Bonus	State machine (CourseRequest); system sequence diagram (alternative flows)
Appendix	PlantUML sources (also supplied as .puml files)

D1. Use case list, actors and traceability

1.1 Use case list with MoSCoW priority

Use case IDs match the team's presentation (slide 11), with two changes: 'Search by course' is merged into View Blocked Students (UC-07), and 'Log In' (UC-01) was added because every role needs it.

ID	Use case	Primary actor	MoSCoW	Description
UC-01	Log In	Student, Advisor, Registrar	Must	User signs in with username and password and lands on the page for their role.
UC-02	Request Course	Student	Must	Student makes one course-wide request for a full course instead of waitlisting each section.
UC-03	State Acceptable Times	Student	Must	Student picks 1-5 section times they can attend; included in UC-02.
UC-04	View Queue Position	Student	Must	Student sees queue position and status of each request.
UC-05	Respond to Offer	Student	Must	Student accepts or declines a seat offer before its deadline.
UC-06	Cancel Request	Student	Should	Student withdraws an active request and leaves the queue.
UC-07	View Blocked Students	Advisor	Should	Advisor sees which advisees are waiting on a full course.
UC-08	Export Blocked-Student List	Advisor	Should	Advisor downloads the blocked-student list as CSV or PDF.
UC-09	Import Registration Data	Registrar	Must	Registrar uploads CSV files for courses, sections and students.
UC-10	Validate Import	(included in UC-09)	Must	System checks headers, types and keys and rejects bad files without changing data.
UC-11	Configure Priority Policy	Registrar	Must	Registrar chooses FIFO, seniority or delay-aware ordering.
UC-12	Generate Offer List	Registrar	Must	When seats open, system builds an ordered offer list; registrar reviews, exports and applies it in Banner.
UC-13	View Demand vs Seats	Registrar	Must	Dashboard of unique demand against open seats for each course.
UC-14	Export Report	Registrar	Should	Registrar exports the demand report as PDF or CSV.
UC-15	Manage Users	Registrar / Admin	Should	Admin creates, disables and assigns roles to accounts.
UC-16	Send Notification	System (E-mail Service)	Must	System e-mails and shows in-app notices for offers and status changes.
UC-17	Expire Offers	System (Scheduler)	Must	Scheduler expires unanswered offers after the deadline and re-offers the seat.
UC-18	Record Audit Log	System	Must	System records who did what and when for key actions.

Won't have (this project): live Banner integration or write-back; automatic enrollment; payment or financial aid; schedule building; mobile native app. Reason: out of scope for a two-person team in 32 weeks and not needed to test the idea.

Count: 18 use cases: 13 Must, 5 Should, 0 Could.

1.2 Actor cards

Student

Field	Content
Actor	Student
Description	Effat student (for example Noura, CS year 3) who needs a seat in a full course.
Triggers / Interests	Triggers: course is full; registration period; offer e-mail. Interests: one request per course, fair order, clear status.
Challenges	Does not know position or chance; limited time to answer an offer; many courses at once.
Notes	Uses UC-01, 02, 03, 04, 05, 06.

Advisor

Field	Content
Actor	Advisor
Description	Faculty advisor (for example Dr. Reem) who guides a group of students.

Field	Content
Triggers / Interests	Triggers: advising week; student asks for help. Interests: know which advisees are blocked, keep plans on track.
Challenges	Sees only own advisees; no access to offers or policy.
Notes	Uses UC-01, 07, 08.

Registrar / Admin

Field	Content
Actor	Registrar / Admin
Description	Registrar analyst (for example Ms. Lama) who manages sections, imports and offer lists; also manages accounts.
Triggers / Interests	Triggers: new term data; seat opens; add/drop deadline. Interests: unique demand, fair and traceable decisions, less manual work.
Challenges	Must apply offers in Banner by hand; data comes only from CSV; responsible for correct policy.
Notes	Uses UC-01, 09, 11, 12, 13, 14, 15.

Scheduler (system timer)

Field	Content
Actor	Scheduler (system timer)
Description	Internal timer that runs every 5 minutes.
Triggers / Interests	Trigger: clock. Interest: no seat stays unused after an offer expires.
Challenges	Must not expire an offer early; must run even if nobody is logged in.
Notes	Starts UC-17.

E-mail Service

Field	Content
Actor	E-mail Service
Description	External mail server used to send notices.
Triggers / Interests	Trigger: UC-16 sends a message. Interest: valid recipients and rate limits.
Challenges	Can be down or slow; messages can bounce.
Notes	Receives requests from UC-16.

Banner (files only)

Field	Content
Actor	Banner (files only)
Description	Effat's student information system, outside our system.
Triggers / Interests	Trigger: registrar exports/imports CSV files. Interest: only official data and staff actions change Banner.
Challenges	No live link; file formats may change; staff must re-enter accepted offers.
Notes	File exchange with UC-09 and UC-12.

1.3 Traceability: use case, user story, requirement

Generated from the same master data as the Lab 3 document, so the IDs agree.

UC	Name	User stories	Requirements (REQ-xx)
UC-01	Log In	US-05	01, 02, 31, 32
UC-02	Request Course	US-01	03, 04, 05, 33
UC-03	State Acceptable Times	US-01	03, 04, 05, 33
UC-04	View Queue Position	US-02	06, 07, 30, 34
UC-05	Respond to Offer	US-03	08, 09, 10
UC-06	Cancel Request	US-04	11
UC-07	View Blocked Students	US-06	12, 13
UC-08	Export Blocked-Student List	US-07	14
UC-09	Import Registration Data	US-08	15, 16, 17, 35, 37
UC-10	Validate Import	US-08	15, 16, 17, 35, 37
UC-11	Configure Priority Policy	US-09	18
UC-12	Generate Offer List	US-10, US-17	19, 20, 21, 22, 38
UC-13	View Demand vs Seats	US-11	23, 24
UC-14	Export Report	US-12	25
UC-15	Manage Users	US-13	26
UC-16	Send Notification	US-03, US-14	08, 09, 10, 27
UC-17	Expire Offers	US-15	28
UC-18	Record Audit Log	US-16	29, 36

D2. Fully dressed use cases

UC-12 Generate Offer List (most complex)

Field	Content
Use Case	UC-12 Generate Offer List
Activity Name	Offer open seats to waiting students in policy order
Primary Actor	Registrar
Stakeholders and Interests	Registrar: a correct, fair list produced quickly. Student: fair order and enough time to answer. Advisor: students blocked by the course move forward. Registrar office staff: a clear export to apply in Banner. E-mail Service: valid recipients. Banner: changed only by staff.
Preconditions	Registrar is logged in (UC-01). Data was imported successfully (UC-09). A priority policy is configured (UC-11). The section has open seats (capacity minus enrolled is greater than 0).
Postconditions	Success: up to N offers exist with status Offered and a deadline; the matching requests are set to Offered; each student has been notified or the failure is logged; audit entries are saved; the offer list can be exported. Failure: no offers are created and no request changes.
Main Scenario	1. Registrar selects a term, course and section and chooses 'Generate offers'. 2. System reads capacity and enrolled count and calculates the open seats N. 3. System loads the Waiting requests for the course. 4. System orders the requests with the active priority policy (FIFO, seniority or delay-aware). 5. System removes requests whose acceptable times do not fit the section. 6. System creates up to N draft offers, each with a deadline (default 48 hours). 7. System shows the ordered draft list to the registrar. 8. Registrar reviews the list and confirms. 9. System sets the offers to Offered and the requests to Offered. 10. System sends a notification to each student (UC-16, included). 11. System records audit entries (UC-18, included). 12. Registrar exports the list as CSV and, when students accept, applies the seats in Banner and marks each offer 'Applied in Banner' (US-17).
Extensions	2a. No open seats: system shows 'No open seats' and the use case ends. 3a. No Waiting requests: system shows 'No waiting students' and the use case ends. 5a. No request fits the section times: system shows an empty list with the reason; registrar may choose another section. 8a. Registrar changes the order: system requires a reason of at least 10 characters and saves it in the audit log (REQ-20); continue at step 8. 8b. Registrar cancels: system discards the drafts; no change is saved. 10a. E-mail fails: system retries up to 3 times at 5-minute intervals and logs the failure; the in-app notice is still shown (REQ-27). *a. System error at any step: no offers are saved; system shows an error and logs it.
Special Requirements	List generated in 10 s or less for 200 requests (REQ-19); export in 5 s or less (REQ-21); position equals offer-list rank (REQ-07); every action audited (REQ-29).
Constraints	The system cannot enroll students in Banner. Seat data is only as fresh as the last import.
Open Issues	How often will the Registrar re-import seat counts? Does Effat want seniority or delay-aware as the default policy? Must the advisor be notified when a student receives an offer? (to ask in interviews)

UC-02 Request Course (most frequent)

Field	Content
Use Case	UC-02 Request Course
Activity Name	Submit one course-wide waitlist request
Primary Actor	Student
Stakeholders and Interests	Student: one request instead of one per section. Registrar: one request per student per course so demand is counted once. Advisor: sees real blocked students.
Preconditions	Student is logged in (UC-01). The course is in the latest import. The student has no active request for this course.
Postconditions	Success: a request with status Waiting exists with 1 to 5 acceptable times and a queue position. Failure: no request is saved.

Field	Content
Main Scenario	1. Student opens 'Request a course' and searches for the course. 2. System shows the course with its sections and times. 3. Student chooses 1 to 5 acceptable times (UC-03, included). 4. Student submits the request. 5. System checks that no active request exists for this student and course. 6. System saves the request as Waiting and calculates the queue position with the active policy. 7. System shows a confirmation with the position and status.
Extensions	2a. Course not found: it is not in the import; system says so and offers a new search. 3a. No time chosen: system asks for at least 1 time. 3b. More than 5 times chosen: system asks the student to remove some. 5a. Active request exists: system rejects the duplicate within 2 s and shows the existing request (REQ-04). *a. Session expired: student logs in again and the form is restored.
Special Requirements	90% of test users finish in 60 s or less (REQ-03); works from 360 px to 1920 px wide (REQ-34); only imported courses are listed (REQ-05).
Constraints	The request does not enroll the student; it only places them in the queue.
Open Issues	Should prerequisites be checked? (Not in scope now; to ask the Registrar.) Maximum number of active requests per student?

Brief descriptions of all use cases

ID	Use case	Precondition	Postcondition
UC-01	Log In	Valid account exists	User is on their role's home page
UC-02	Request Course	Logged in; course imported; no active request	Request Waiting with times and position
UC-03	State Acceptable Times	Student is making a request	1 to 5 times stored with the request
UC-04	View Queue Position	Student has at least one request	Student sees position and status
UC-05	Respond to Offer	Student has an open offer before its deadline	Request Accepted or Declined; queue updated
UC-06	Cancel Request	Request is Waiting or Offered	Request Cancelled; others re-ordered
UC-07	View Blocked Students	Advisor logged in; has advisees	List of blocked advisees shown
UC-08	Export Blocked-Student List	Blocked list is displayed	CSV or PDF file produced
UC-09	Import Registration Data	Registrar logged in; CSV files ready	New data stored, or nothing changed
UC-10	Validate Import	A file was uploaded	File accepted, or rejected with line errors
UC-11	Configure Priority Policy	Registrar logged in	Policy saved; queues re-ordered
UC-12	Generate Offer List	Open seats; policy set	Offers created and students notified
UC-13	View Demand vs Seats	Data imported	Unique demand vs seats shown per course
UC-14	Export Report	Dashboard data exists	PDF or CSV report produced
UC-15	Manage Users	Admin logged in	Account created, disabled or role changed
UC-16	Send Notification	An offer or status change occurred	E-mail sent and in-app notice shown, or failure logged
UC-17	Expire Offers	Offers past deadline exist	Offers Expired; seats re-offered
UC-18	Record Audit Log	A key action happened	Entry with user, time and action stored

D3. Use case diagram

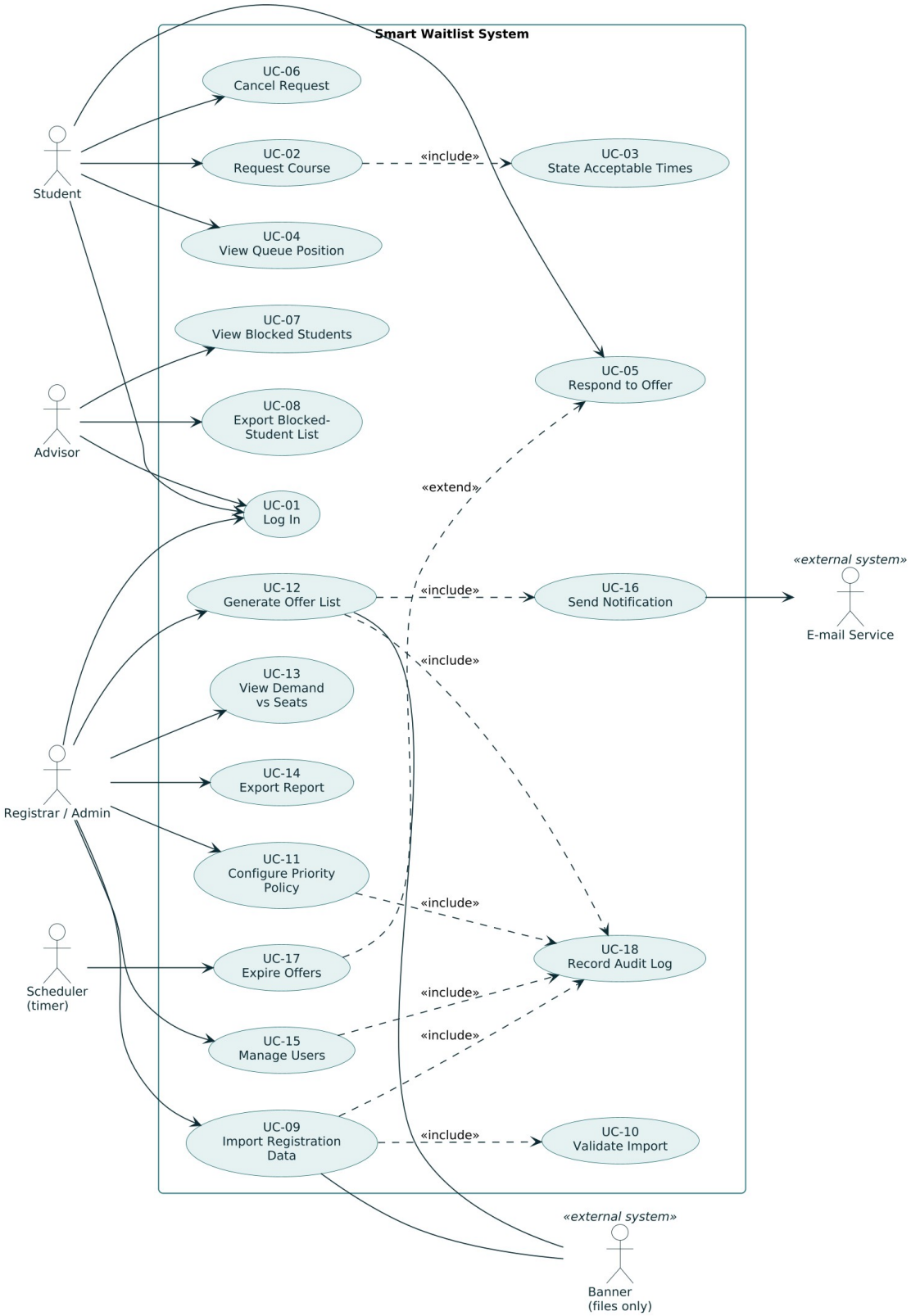


Figure 1. Use case diagram. Solid lines: actor participation. Dotted arrows: <<include>> (UC-02 to UC-03, UC-09 to UC-10, UC-12 to UC-16, and UC-09, 11, 12, 15 to UC-18) and <<extend>> (UC-17 extends UC-05). E-mail Service and Banner are external-system actors.

D4. Activity diagram: UC-12 Generate Offer List

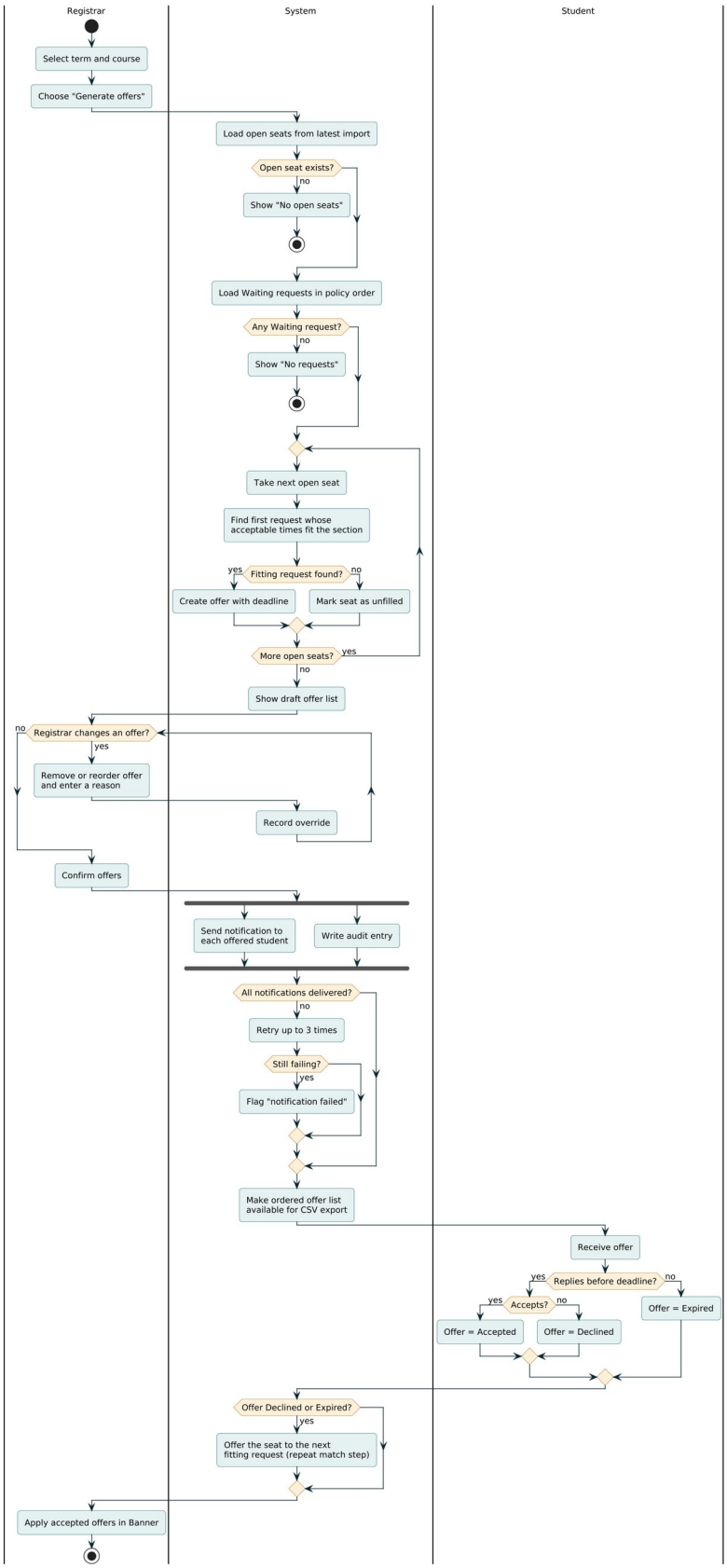


Figure 2. Activity diagram with swimlanes (Registrar, System, Student), guards, loops, a fork for notifications and audit, and the offer outcomes accepted, declined and expired.

D5. Sequence diagram: UC-12 main success scenario

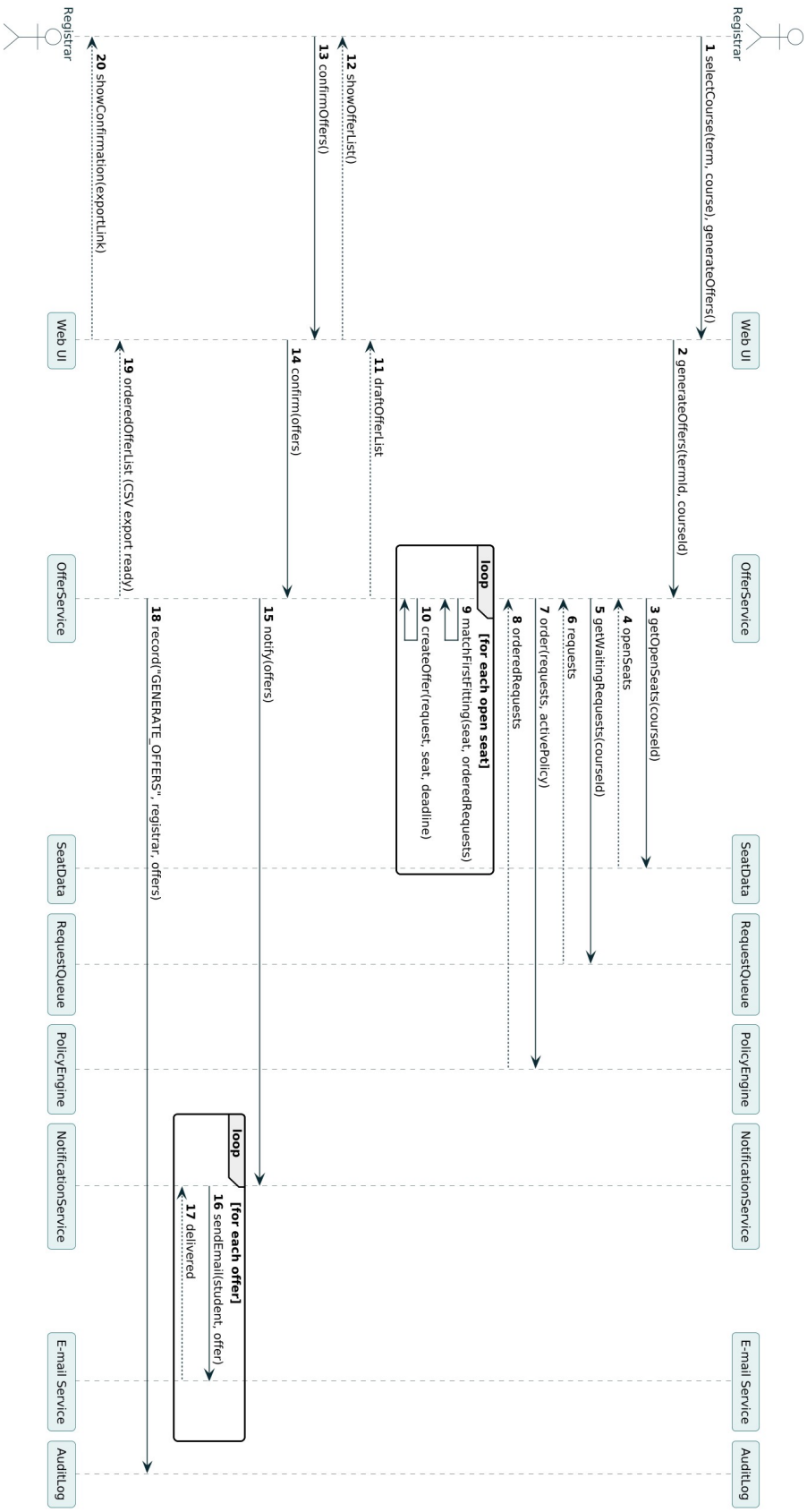


Figure 3 (rotated 90 degrees clockwise; turn the page to read). Sequence diagram of the 12-step main scenario (20 messages). Participants match the domain model classes and services.

D6. Domain model

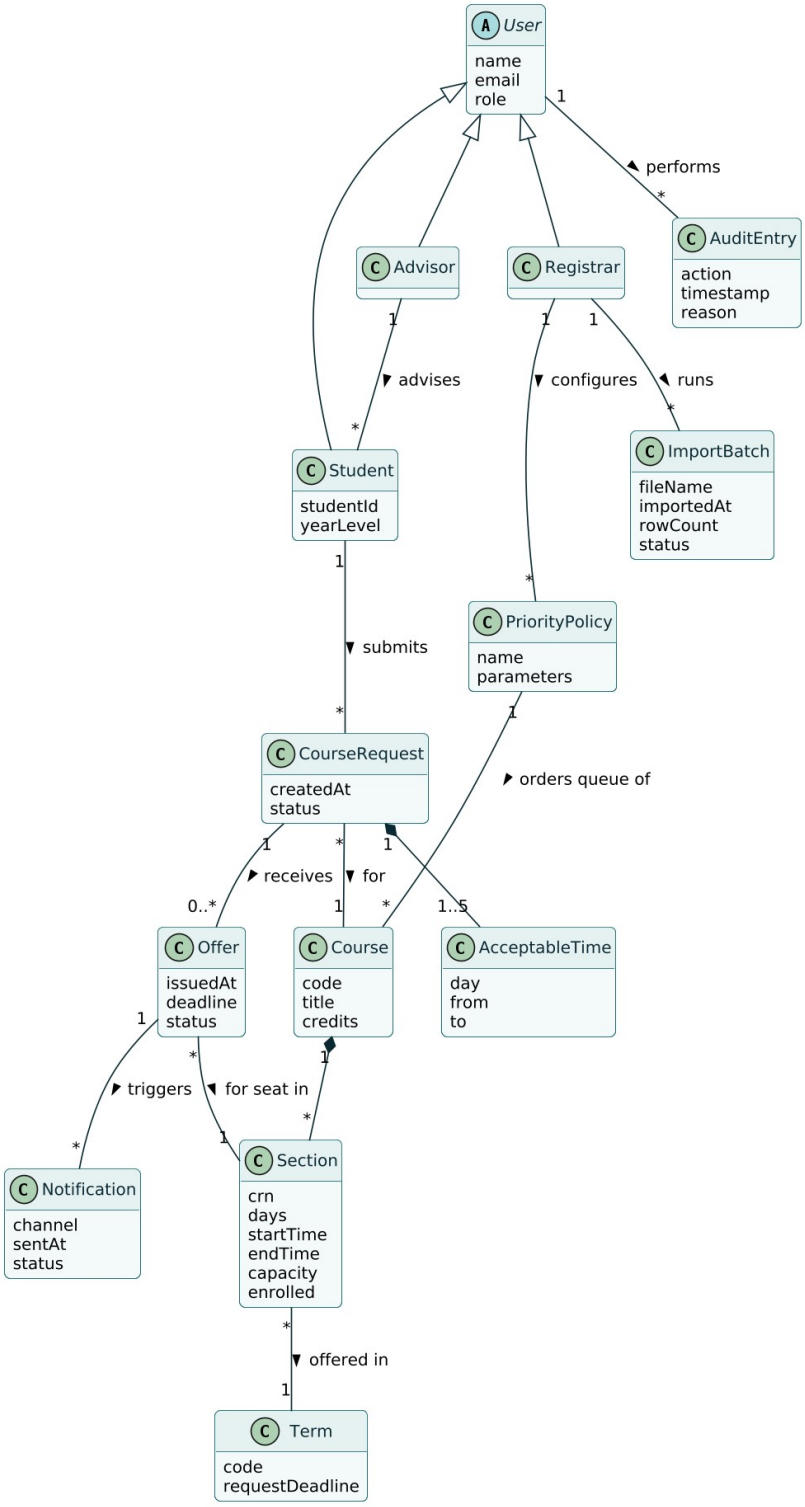


Figure 4. Domain model. AcceptableTime is composed in CourseRequest (1 to 5). Student, Advisor and Registrar specialise User.

Domain notes: a Student may hold many CourseRequests but only one active request per Course (REQ-04). An Offer belongs to one CourseRequest and one Section. PriorityPolicy is chosen per Term. AuditEntry records the User and the action.

Bonus E1. State machine: CourseRequest

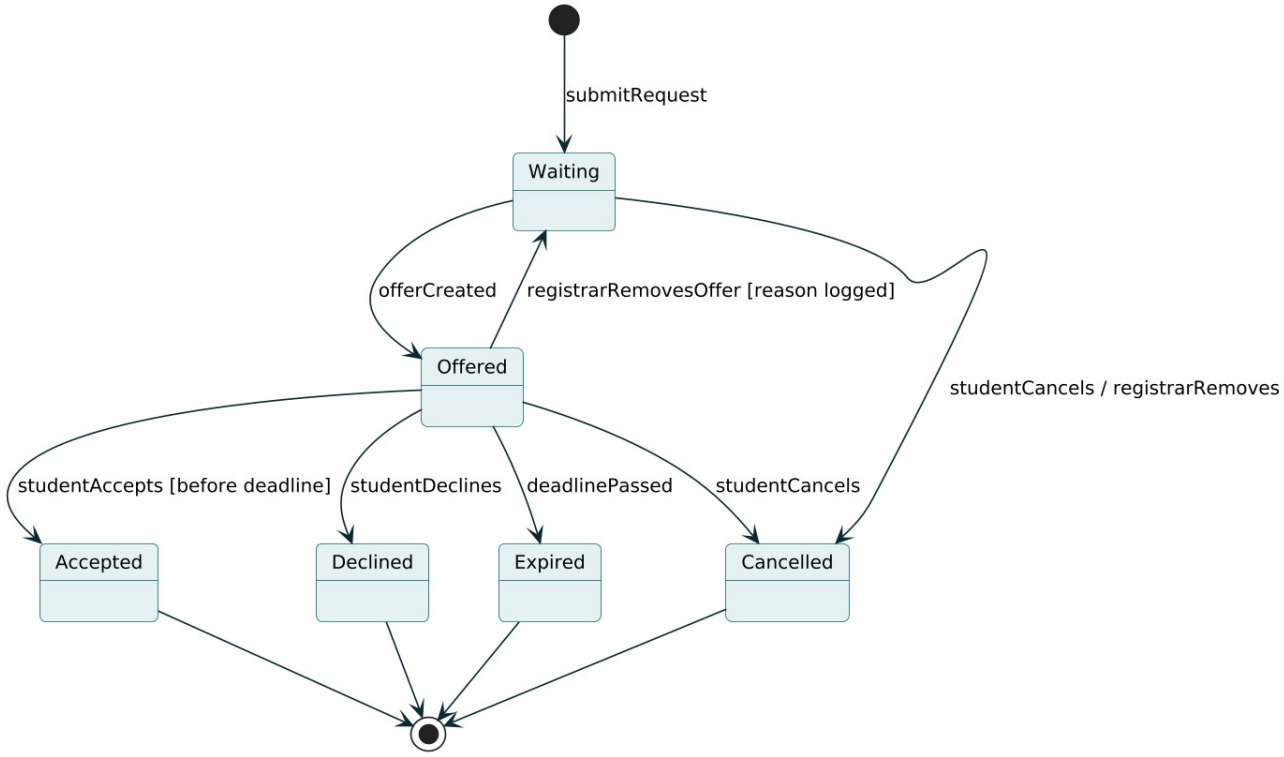


Figure 5. States of a CourseRequest: Waiting, Offered, Accepted, Declined, Expired, Cancelled. A registrar removing an offer returns the request from Offered to Waiting.

Bonus E2. System sequence diagram: alternative flows

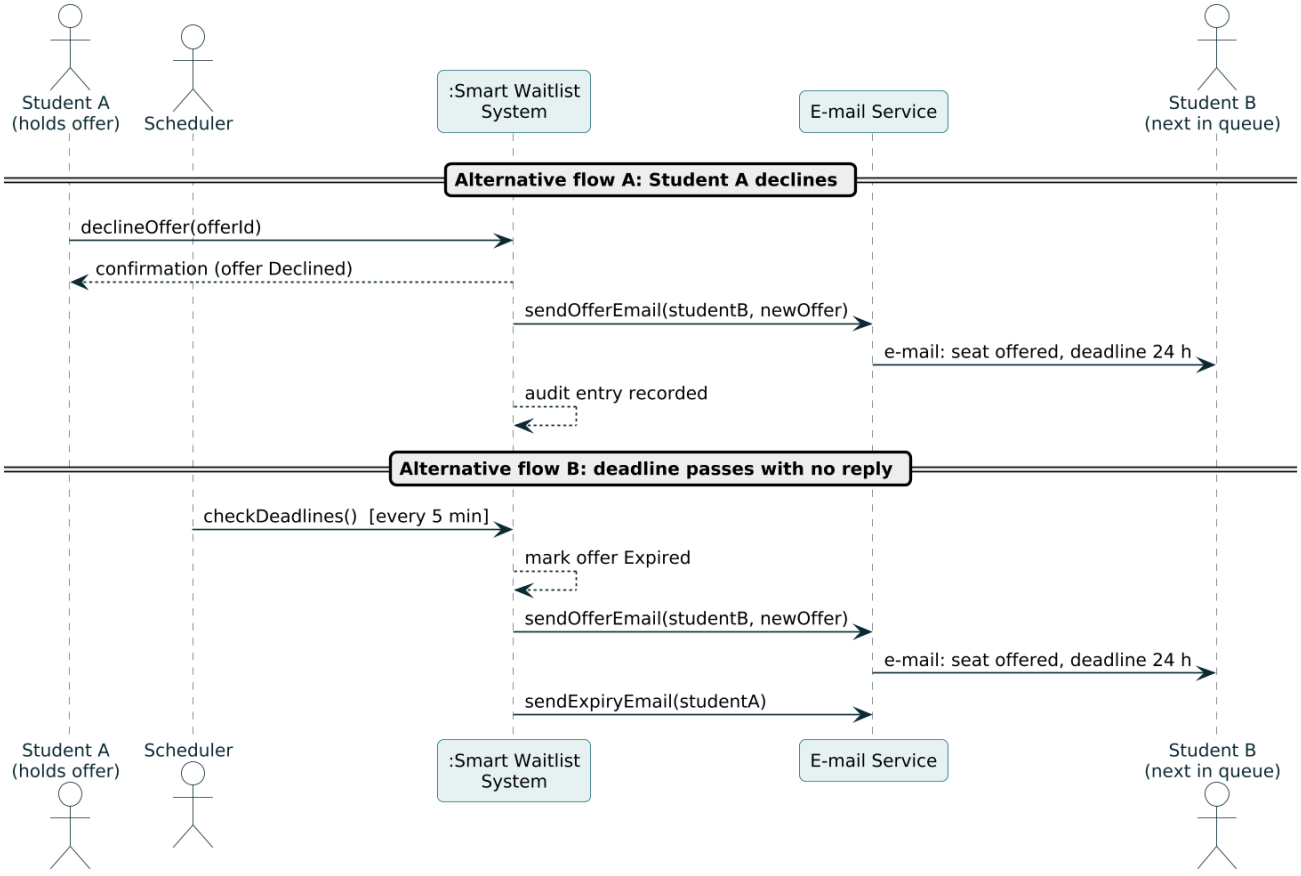


Figure 6. Alternative flow A: the student declines and the next student is e-mailed. Alternative flow B: the scheduler finds an expired offer (checked every 5 minutes) and the next student is e-mailed.

D7. Consistency checklist and peer review

7.1 Consistency checklist (self-check)

Check	Status
Every use case in D1 appears in the use case diagram (UC-01 to UC-18)	Done
Every actor in the diagram has an actor card	Done
Every use case has at least one user story and one requirement (traceability matrix)	Done
Include and extend relations are the same in D2, D3 and D4	Done
Main scenario steps in D2 match activity (D4) and sequence (D5) for UC-12	Done
Classes and services in D5 exist in the domain model (D6) or are named services	Done
States in E1 equal the request statuses used in REQ-06 and the Lab 3 document	Done
Use case names follow verb + object	Done
Deck (presentation) use case list matches D1 except the two noted changes	Done
Peer review by another team	Pending: sheet below left blank for the reviewing team

7.2 Peer-review sheet (to be filled by another team)

Reviewing team: _____ Date: _____ Reviewed team: Smart Waitlist (Mohammad Altazi, Omar AlMutairi)

Criterion	Score (1-5)	Comments / suggested change
Use cases are clear, complete and named with verb + object		
Actors are correct and not missing (including external systems)		
Use case diagram is readable and relations (include/extend) are used correctly		
Fully dressed use cases have realistic extensions and special requirements		
Activity diagram shows decisions, loops and swimlanes correctly		
Sequence diagram matches the main scenario		
Domain model has correct classes, associations and multiplicities		
Diagrams, IDs and the requirements are consistent		

Overall comment: _____

Appendix A. PlantUML sources

The same files are supplied separately in the folder uml/puml. Render with: java -jar plantuml.jar -tpng file.puml

common.iuml

```
skinparam shadowing false
skinparam defaultFontName DejaVu Sans
skinparam defaultFontSize 12
skinparam dpi 160
skinparam roundcorner 8
skinparam ArrowColor #0B2A33
skinparam usecase {
  BackgroundColor #E6F1F2
  BorderColor #1B6B73
  FontColor #0B2A33
}
skinparam actor {
  BackgroundColor #FFFFFF
  BorderColor #0B2A33
  FontColor #0B2A33
}
skinparam rectangle {
  BackgroundColor #FFFFFF
  BorderColor #1B6B73
}
skinparam class {
  BackgroundColor #F4F9F9
  BorderColor #1B6B73
  FontColor #0B2A33
  HeaderBackgroundColor #E6F1F2
}
skinparam sequence {
  ParticipantBackgroundColor #E6F1F2
  ParticipantBorderColor #1B6B73
  LifeLineBorderColor #6B7C80
  ActorBorderColor #0B2A33
}
skinparam activity {
  BackgroundColor #E6F1F2
  BorderColor #1B6B73
  DiamondBackgroundColor #FDF1DC
  DiamondBorderColor #BA7517
}
skinparam state {
  BackgroundColor #E6F1F2
  BorderColor #1B6B73
}
```

d3_use_case_diagram.puml

```
@startuml
!include common.iuml
left to right direction
actor Student
actor Advisor
actor "Registrar / Admin" as Reg
actor "Scheduler\n(timer)" as Timer
actor "E-mail Service" as Mail <<external system>>
actor "Banner\n(files only)" as Banner <<external system>>

rectangle "Smart Waitlist System" {
  usecase "UC-01\nLog In" as UC01
  usecase "UC-02\nRequest Course" as UC02
  usecase "UC-03\nState Acceptable Times" as UC03
  usecase "UC-04\nView Queue Position" as UC04
  usecase "UC-05\nRespond to Offer" as UC05
  usecase "UC-06\nCancel Request" as UC06
  usecase "UC-07\nView Blocked Students" as UC07
  usecase "UC-08\nExport Blocked-\nStudent List" as UC08
  usecase "UC-09\nImport Registration\nData" as UC09
  usecase "UC-10\nValidate Import" as UC10
  usecase "UC-11\nConfigure Priority\nPolicy" as UC11
  usecase "UC-12\nGenerate Offer List" as UC12
  usecase "UC-13\nView Demand\nvs Seats" as UC13
  usecase "UC-14\nExport Report" as UC14
  usecase "UC-15\nManage Users" as UC15
  usecase "UC-16\nSend Notification" as UC16
  usecase "UC-17\nExpire Offers" as UC17
  usecase "UC-18\nRecord Audit Log" as UC18
}
```

```
Student --> UC01
Advisor --> UC01
Reg --> UC01
Student --> UC02
Student --> UC04
Student --> UC05
Student --> UC06
Advisor --> UC07
Advisor --> UC08
Reg --> UC09
Reg --> UC11
Reg --> UC12
Reg --> UC13
Reg --> UC14
Reg --> UC15
Timer --> UC17

UC02 ..> UC03 : <<include>>
UC09 ..> UC10 : <<include>>
UC12 ..> UC16 : <<include>>
UC09 ..> UC18 : <<include>>
UC11 ..> UC18 : <<include>>
UC12 ..> UC18 : <<include>>
UC15 ..> UC18 : <<include>>
UC17 ..> UC05 : <<extend>>
UC16 --> Mail
UC09 -- Banner
UC12 -- Banner
@enduml
```

d4_activity_diagram.puml

```
@startuml
!include common.iuml
|Registrar|
start
:Select term and course;
:Choose "Generate offers";
|System|
:Load open seats from latest import;
if (Open seat exists?) then (no)
  :Show "No open seats";
  stop
endif
:Load Waiting requests in policy order;
if (Any Waiting request?) then (no)
  :Show "No requests";
  stop
endif
repeat
  :Take next open seat;
  :Find first request whose
  acceptable times fit the section;
  if (Fitting request found?) then (yes)
    :Create offer with deadline;
  else (no)
    :Mark seat as unfilled;
  endif
repeat while (More open seats?) is (yes) not (no)
:Show draft offer list;
|Registrar|
while (Registrar changes an offer?) is (yes)
  :Remove or reorder offer
  and enter a reason;
  |System|
  :Record override;
  |Registrar|
endwhile (no)
:Confirm offers;
|System|
fork
  :Send notification to
  each offered student;
fork again
  :Write audit entry;
end fork
if (All notifications delivered?) then (no)
  :Retry up to 3 times;
  if (Still failing?) then (yes)
    :Flag "notification failed";
  endif
endif
:Make ordered offer list
available for CSV export;
|Student|
```

```
:Receive offer;
if (Replies before deadline?) then (yes)
  if (Accepts?) then (yes)
    :Offer = Accepted;
  else (no)
    :Offer = Declined;
  endif
else (no)
  :Offer = Expired;
endif
|System|
if (Offer Declined or Expired?) then (yes)
  :Offer the seat to the next
  fitting request (repeat match step);
endif
|Registrar|
:Apply accepted offers in Banner;
stop
@enduml
```

d5_sequence_diagram.puml

```
@startuml
!include common.iuml
autonumber
actor Registrar
participant "Web UI" as UI
participant OfferService
participant "SeatData" as Seats
participant "RequestQueue" as Queue
participant PolicyEngine
participant NotificationService as Notif
participant "E-mail Service" as Mail
participant AuditLog

Registrar -> UI : selectCourse(term, course), generateOffers()
UI -> OfferService : generateOffers(termId, courseId)
OfferService -> Seats : getOpenSeats(courseId)
Seats --> OfferService : openSeats
OfferService -> Queue : getWaitingRequests(courseId)
Queue --> OfferService : requests
OfferService -> PolicyEngine : order(requests, activePolicy)
PolicyEngine --> OfferService : orderedRequests
loop for each open seat
  OfferService -> OfferService : matchFirstFitting(seat, orderedRequests)
  OfferService -> OfferService : createOffer(request, seat, deadline)
end
OfferService --> UI : draftOfferList
UI --> Registrar : showOfferList()
Registrar -> UI : confirmOffers()
UI -> OfferService : confirm(offers)
OfferService -> Notif : notify(offers)
loop for each offer
  Notif -> Mail : sendEmail(student, offer)
  Mail --> Notif : delivered
end
OfferService -> AuditLog : record("GENERATE_OFFERS", registrar, offers)
OfferService --> UI : orderedOfferList (CSV export ready)
UI --> Registrar : showConfirmation(exportLink)
@enduml
```

d6_domain_model.puml

```
@startuml
!include common.iuml
hide empty methods
abstract class User {
  name
  email
  role
}
class Student {
  studentId
  yearLevel
}
class Advisor
class Registrar
class Course {
  code
  title
  credits
}
class Term {
  code
  requestDeadline
```

```
}
class Section {
  crn
  days
  startTime
  endTime
  capacity
  enrolled
}
class CourseRequest {
  createdAt
  status
}
class AcceptableTime {
  day
  from
  to
}
class Offer {
  issuedAt
  deadline
  status
}
class PriorityPolicy {
  name
  parameters
}
class Notification {
  channel
  sentAt
  status
}
class ImportBatch {
  fileName
  importedAt
  rowCount
  status
}
class AuditEntry {
  action
  timestamp
  reason
}

User <|-- Student
User <|-- Advisor
User <|-- Registrar
Advisor "1" -- "*" Student : advises >
Student "1" -- "*" CourseRequest : submits >
CourseRequest "*" -- "1" Course : for >
CourseRequest "1" *-- "1..5" AcceptableTime
Course "1" *-- "*" Section
Section "*" -- "1" Term : offered in >
CourseRequest "1" -- "0..*" Offer : receives >
Offer "*" -- "1" Section : for seat in >
PriorityPolicy "1" -- "*" Course : orders queue of >
Offer "1" -- "*" Notification : triggers >
Registrar "1" -- "*" ImportBatch : runs >
Registrar "1" -- "*" PriorityPolicy : configures >
User "1" -- "*" AuditEntry : performs >
@enduml
```

e1_state_machine.puml

```
@startuml
!include common.iuml
[*] --> Waiting : submitRequest
Waiting --> Offered : offerCreated
Waiting --> Cancelled : studentCancels / registrarRemoves
Offered --> Accepted : studentAccepts [before deadline]
Offered --> Declined : studentDeclines
Offered --> Expired : deadlinePassed
Offered --> Cancelled : studentCancels
Offered --> Waiting : registrarRemovesOffer [reason logged]
Accepted --> [*]
Declined --> [*]
Expired --> [*]
Cancelled --> [*]
@enduml
```

e2_system_sequence_alt.puml

```
@startuml
!include common.iuml
actor "Student A\n(holds offer)" as A
```

```
actor Scheduler
participant ":Smart Waitlist\nSystem" as SYS
participant "E-mail Service" as Mail
actor "Student B\n(next in queue)" as B

== Alternative flow A: Student A declines ==
A -> SYS : declineOffer(offerId)
SYS --> A : confirmation (offer Declined)
SYS -> Mail : sendOfferEmail(studentB, newOffer)
Mail -> B : e-mail: seat offered, deadline 24 h
SYS --> SYS : audit entry recorded

== Alternative flow B: deadline passes with no reply ==
Scheduler -> SYS : checkDeadlines()  [every 5 min]
SYS --> SYS : mark offer Expired
SYS -> Mail : sendOfferEmail(studentB, newOffer)
Mail -> B : e-mail: seat offered, deadline 24 h
SYS -> Mail : sendExpiryEmail(studentA)
@enduml
```